

UML

Nombre

David Alejandro Rosales Sarria

Instructor

Luis Fernando Sánchez Pérez

Ficha

3169892

Servicio Nacional de Aprendizaje SENA

Centro De Tecnología De La Manufactura Avanzada CTMA

Análisis y Desarrollo De Software

2025

Guía de Actividad 2 – UML

Actividad 1: Diagrama de Casos de Uso en UML

Solución

Consulte cómo se crea un Diagrama de Casos de Uso en UML y cuáles son los elementos que lo componen, realice un diagrama para describir un proceso que identifique en su contexto (por ejemplo: el proceso para ir de compras al supermercado, un proceso de matrícula, el proceso para hacer un saque en un partido de tenis, etc.).

R: Un diagrama de casos de uso en UML (Lenguaje Unificado de Modelado) es una herramienta visual que representa las interacciones entre los usuarios (actores) y un sistema, mostrando las funcionalidades que el sistema ofrece desde la perspectiva del usuario. Es especialmente útil en las etapas iniciales del desarrollo de software para capturar y comunicar los requisitos funcionales.

Elementos principales de un diagrama de casos de uso

1. Actor:

- Representa a una entidad externa que interactúa con el sistema, como un usuario, otro sistema o un dispositivo.
- Se simboliza con una figura de persona estilizada.
- Puede ser activo (inicia la interacción) o pasivo (recibe información del sistema) .

2. Caso de uso:

- Describe una funcionalidad específica que el sistema proporciona a los actores.
- Se representa mediante una elipse que contiene el nombre del caso de uso, generalmente utilizando verbos en infinitivo, como "Registrar usuario" o "Procesar pago"

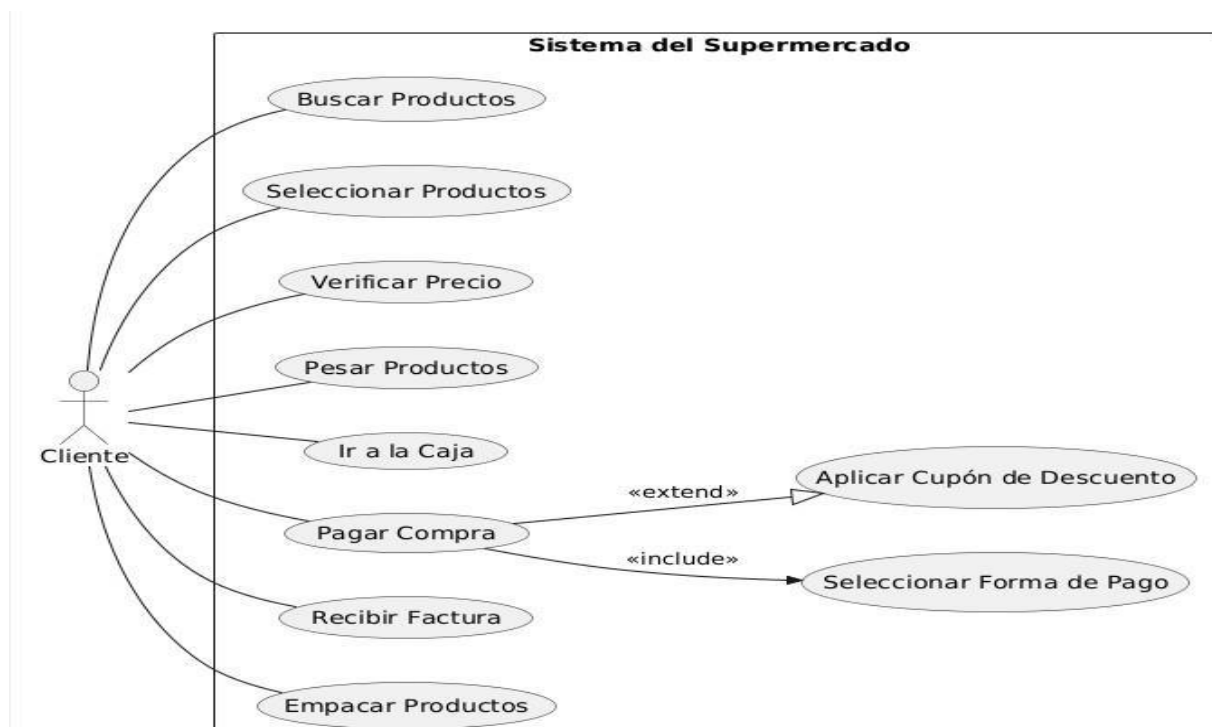
3. Relaciones:

- **Asociación:** Línea que conecta un actor con un caso de uso, indicando que el actor participa en ese caso.
- **Inclusión (<<include>>):** Indica que un caso de uso incorpora explícitamente el comportamiento de otro. Se representa con una línea punteada y una flecha desde el caso de uso principal al incluido.
- **Extensión (<<extend>>):** Muestra que un caso de uso puede extender el comportamiento de otro bajo ciertas condiciones. Se representa con una línea punteada y una flecha desde el caso de uso extendido al principal.
- **Generalización:** Define una relación jerárquica entre actores o casos de uso, donde uno hereda el comportamiento de otro. Se representa con una línea continua y una flecha hacia el elemento generalizado

4. Sistema (límite del sistema):

- Representado por un rectángulo que delimita el alcance del sistema.
- Los casos de uso se colocan dentro de este rectángulo, mientras que los actores se sitúan fuera de él.

Diagrama de proceso de matrícula



Actividad 2: Diagrama de Clases en UML

Un diagrama de clases en UML (Lenguaje Unificado de Modelado) es una representación visual de la estructura estática de un sistema. Describe las clases del sistema, sus atributos, sus métodos y las relaciones entre ellas. Estos son los elementos principales que lo componen:

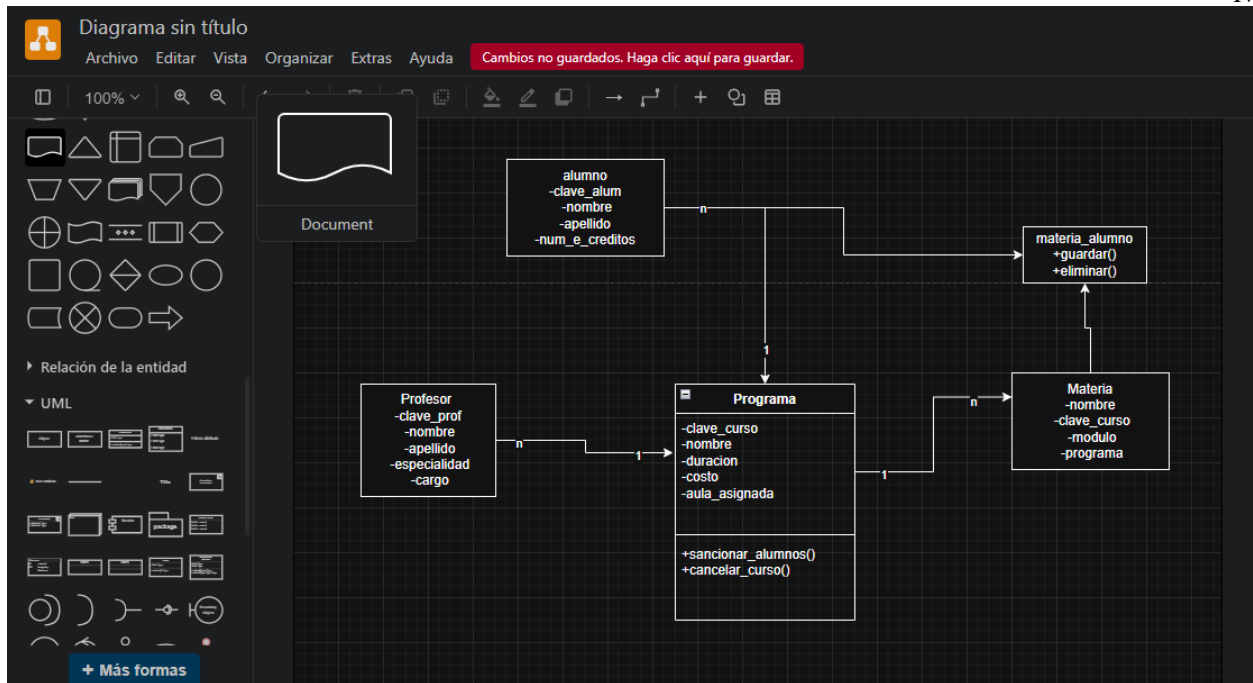
1. Clases:

- Se representan como un rectángulo dividido en tres secciones:
 - **Nombre de la clase:** En la parte superior, siempre obligatorio. Puede estar en cursiva si la clase es abstracta.
 - **Atributos:** En la sección central. Representan las características o propiedades de la clase (variables o campos). Cada atributo se lista con su nombre, tipo de dato y, opcionalmente, su visibilidad (público +, privado -, protegido #, de paquete ~, o derivado /).
 - **Métodos (u Operaciones):** En la sección inferior. Representan las acciones o comportamientos que la clase puede realizar. Cada método se lista con su nombre, parámetros (con sus tipos de dato) y el tipo de retorno, además de su visibilidad.

2. Relaciones: Son las conexiones entre las clases, indicando cómo interactúan entre sí. Los tipos más comunes son:

- **Asociación:** Representa una conexión semántica entre dos o más clases. Se dibuja con una línea sólida entre las clases. Puede tener:

- **Multiplicidad:** Indica cuántas instancias de una clase están relacionadas con instancias de otra clase (ej. 1 para uno, * para muchos, 0..1 para cero o uno, 1..* para uno o muchos).
 - **Roles:** Nombre que describe el papel de una clase en la asociación.
 - **Navegabilidad:** Una flecha que indica la dirección en la que se puede acceder a la clase relacionada.
 - **Herencia (Generalización/Especialización):** Indica una relación "es un tipo de". Una clase (subclase o clase hija) hereda atributos y métodos de otra clase (superclase o clase padre). Se representa con una línea sólida que tiene una flecha hueca apuntando a la superclase.
 - **Agregación:** Un tipo especial de asociación que representa una relación "tiene un" o "parte-todo", donde las "partes" pueden existir independientemente del "todo". Se dibuja con una línea sólida y un rombo hueco en el extremo de la clase "todo".
 - **Composición:** Un tipo fuerte de agregación donde las "partes" no pueden existir sin el "todo". Si el "todo" se elimina, las "partes" también se eliminan. Se dibuja con una línea sólida y un rombo relleno en el extremo de la clase "todo".
 - **Dependencia:** Indica que un cambio en una clase puede afectar a otra clase. Es una relación más débil que la asociación. Se representa con una línea discontinua con una flecha apuntando a la clase de la que se depende.
 - **Realización (Implementación):** Se utiliza cuando una clase implementa una interfaz. Se representa con una línea discontinua con una flecha hueca apuntando a la interfaz.
3. **Otros elementos importantes:**
- **Interfaces:** Son clases abstractas que solo contienen operaciones (no atributos) y definen un contrato que las clases que las implementan deben cumplir. Se representan como una clase con el estereotipo <<interface>> o como un círculo con el nombre de la interfaz.
 - **Tipos de datos:** Representan valores de datos primitivos (como enteros, booleanos, cadenas) o enumeraciones. No pueden tener relaciones con las clases, aunque las clases sí pueden tener relaciones con ellos.
 - **Enumeraciones:** Una lista de valores predefinidos (literales de enumeración).
 - **Paquetes:** Sirven para organizar clasificadores (clases, interfaces, etc.) relacionados en grupos lógicos, mejorando la legibilidad de diagramas complejos. Se representan como una carpeta.
 - **Notas:** Permiten añadir comentarios o información textual al diagrama. Se representan como un rectángulo con una esquina doblada.



Evidencias de la base de datos SQL y NoSQL

Base de datos SQL

Creación de la base de datos y tablas:

```

1 CREATE DATABASE base_de_inscripciones;
2 USE base_de_inscripciones
3
4 CREATE TABLE alumno (
5     id_Alumno int NOT NULL,
6     Nombre varchar(40) NOT NULL,
7     Apellido varchar(40) NOT NULL,
8     Num_e_Creditos int NOT NULL,
9     id_Curso int NOT NULL
10 );
11
12 CREATE TABLE calificaciones (
13     id_Alumno int NOT NULL,
14     id_Materia int NOT NULL,
15     nota decimal(4,2) NOT NULL
16 );
17
18 CREATE TABLE horario (
19     id_Horario int NOT NULL,
20     id_Materia int NOT NULL,
21     dia varchar(15) NOT NULL,
22     Hora_Inicio time NOT NULL,
23     Hora_Fin time NOT NULL
24 );

```

```
26 CREATE TABLE materia (  
27     id_Materia int NOT NULL AUTO_INCREMENT,  
28     Nombre varchar(20) NOT NULL,  
29     Modulo int NOT NULL,  
30     Id_Curso int NOT NULL,  
31     PRIMARY KEY (id_Materia)  
32 );  
33  
34 CREATE TABLE materia_alumno (  
35     id_Alumno int NOT NULL,  
36     id_Materia int NOT NULL,  
37     PRIMARY KEY (id_Alumno, id_Materia)  
38 );  
39  
40 CREATE TABLE profesor (  
41     id_Profesor int NOT NULL,  
42     Nombre varchar(50) NOT NULL,  
43     Apellido varchar(30) NOT NULL,  
44     Especialidad varchar(30) NOT NULL,  
45     Cargo varchar(30) NOT NULL,  
46     id_Curso int NOT NULL  
47 );  
  
49 CREATE TABLE profesor_materia (  
50     id_Profesor int NOT NULL,  
51     id_Materia int NOT NULL  
52 );  
53  
54 CREATE TABLE programa (  
55     id_Curso int NOT NULL,  
56     Nombre varchar(15) NOT NULL,  
57     Duración int NOT NULL,  
58     Aula_Asignada int NOT NULL,  
59     PRIMARY KEY (id_Curso)  
60 );
```

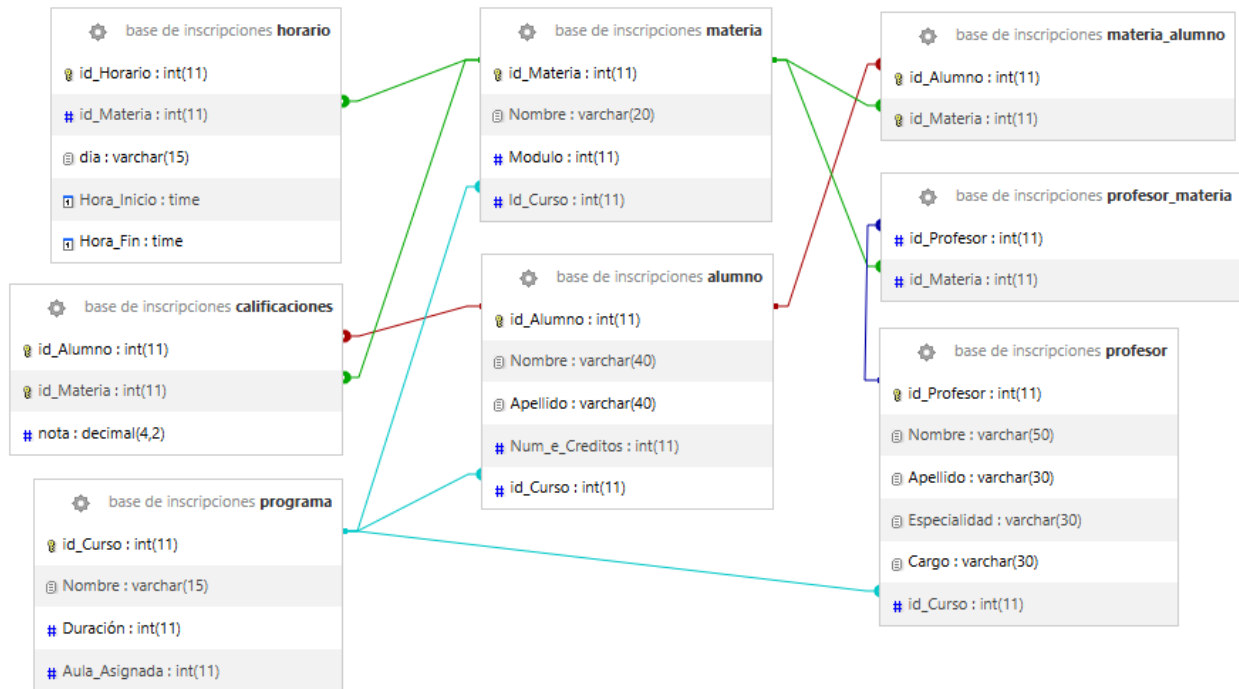
Creación de las PK y las FK:

```
62 ALTER TABLE alumno
63     ADD PRIMARY KEY (id_Alumno),
64     ADD KEY id_Curso (id_Curso);
65
66 ALTER TABLE calificaciones
67     ADD PRIMARY KEY (id_Alumno, id_Materia),
68     ADD KEY id_Materia (id_Materia);
69
70 ALTER TABLE horario
71     ADD PRIMARY KEY (id_Horario),
72     ADD KEY id_Materia (id_Materia);
73
74 ALTER TABLE materia
75     ADD KEY Id_Curso (Id_Curso);
76
77 ALTER TABLE materia_alumno
78     ADD KEY id_Materia (id_Materia);
79
80 ALTER TABLE profesor
81     ADD PRIMARY KEY (id_Profesor),
82     ADD KEY id_Curso (id_Curso);
83
84 ALTER TABLE profesor_materia
85     ADD KEY id_Materia (id_Materia),
86     ADD KEY id_Profesor (id_Profesor);
87
91 ALTER TABLE calificaciones
92     ADD CONSTRAINT calificaciones_ibfk_1 FOREIGN KEY (id_Alumno) REFERENCES alumno (id_Alumno),
93     ADD CONSTRAINT calificaciones_ibfk_2 FOREIGN KEY (id_Materia) REFERENCES materia (id_Materia);
94
95 ALTER TABLE horario
96     ADD CONSTRAINT horario_ibfk_1 FOREIGN KEY (id_Materia) REFERENCES materia (id_Materia);
97
98 ALTER TABLE materia
99     ADD CONSTRAINT materia_ibfk_1 FOREIGN KEY (Id_Curso) REFERENCES programa (id_Curso);
100
101 ALTER TABLE materia_alumno
102     ADD CONSTRAINT materia_alumno_ibfk_1 FOREIGN KEY (id_Alumno) REFERENCES alumno (id_Alumno),
103     ADD CONSTRAINT materia_alumno_ibfk_2 FOREIGN KEY (id_Materia) REFERENCES materia (id_Materia);
104
105 ALTER TABLE profesor
106     ADD CONSTRAINT profesor_ibfk_1 FOREIGN KEY (id_Curso) REFERENCES programa (id_Curso);
107
108 ALTER TABLE profesor_materia
109     ADD CONSTRAINT profesor_materia_ibfk_1 FOREIGN KEY (id_Materia) REFERENCES materia (id_Materia),
110     ADD CONSTRAINT profesor_materia_ibfk_2 FOREIGN KEY (id_Profesor) REFERENCES profesor (id_Profesor);
```

Agregar datos a las tablas:

```
1  INSERT INTO programa (id_Curso, Nombre, Duración, Aula_Asignada) VALUES
2  (1, 'Sistemas', 4, 101),
3  (2, 'Contabilidad', 3, 102);
4
5  INSERT INTO materia (id_Materia, Nombre, Modulo, Id_Curso) VALUES
6  (1, 'Matemáticas', 1, 1),
7  (2, 'Programación', 1, 1),
8  (3, 'Finanzas', 1, 2),
9  (4, 'Contabilidad I', 1, 2);
10
11 INSERT INTO profesor (id_Profesor, Nombre, Apellido, Especialidad, Cargo, id_Curso) VALUES
12 (1, 'Carlos', 'Pérez', 'Matemáticas', 'Titular', 1),
13 (2, 'Luisa', 'Gómez', 'Informática', 'Titular', 1),
14 (3, 'Ana', 'Martínez', 'Finanzas', 'Catedrática', 2);
15
16 INSERT INTO alumno (id_Alumno, Nombre, Apellido, Num_e_Creditos, id_Curso) VALUES
17 (1, 'Juan', 'López', 10, 1),
18 (2, 'María', 'Rodríguez', 8, 2),
19 (3, 'Pedro', 'García', 12, 1),
20 (4, 'Laura', 'Ramírez', 6, 2);
21
22 INSERT INTO materia_alumno (id_Alumno, id_Materia) VALUES
23 (1, 1), (1, 2),
24 (2, 3), (2, 4),
25 (3, 1), (3, 2),
26 (4, 3), (4, 4);
27
28 INSERT INTO calificaciones (id_Alumno, id_Materia, nota) VALUES
29 (1, 1, 4.5), (1, 2, 4.2),
30 (2, 3, 3.8), (2, 4, 4.0),
31 (3, 1, 4.0), (3, 2, 3.9),
32 (4, 3, 4.1), (4, 4, 4.3);
33
34 INSERT INTO profesor_materia (id_Profesor, id_Materia) VALUES
35 (1, 1),
36 (2, 2),
37 (3, 3), (3, 4);
38
39 INSERT INTO horario (id_Horario, id_Materia, dia, Hora_Inicio, Hora_Fin) VALUES
40 (1, 1, 'Lunes', '08:00:00', '10:00:00'),
41 (2, 2, 'Martes', '10:00:00', '12:00:00'),
42 (3, 3, 'Miércoles', '08:00:00', '10:00:00'),
43 (4, 4, 'Jueves', '10:00:00', '12:00:00');
```


Tablas en el modo diseñador:



Demostración de consultas en SQL:

```

1 SELECT
2     alumno.id_Alumno,
3     alumno.Nombre,
4     alumno.Apellido,
5     alumno.Num_e_Creditos,
6     programa.Nombre AS Nombre_Programa
7 FROM alumno
8 INNER JOIN programa ON alumno.id_Curso = programa.id_Curso;

```

id_Alumno	Nombre	Apellido	Num_e_Creditos	Nombre_Programa
1	Juan	López	10	Sistemas
2	María	Rodríguez	8	Contabilidad
3	Pedro	García	12	Sistemas
4	Laura	Ramírez	6	Contabilidad

```

1 SELECT
2   calificaciones.id_Alumno,
3   alumno.Nombre AS Nombre_Alumno,
4   alumno.Apellido AS Apellido_Alumno,
5   calificaciones.id_Materia,
6   materia.Nombre AS Nombre_Materia,
7   calificaciones.nota
8 FROM calificaciones
9 INNER JOIN alumno ON calificaciones.id_Alumno = alumno.id_Alumno
10 INNER JOIN materia ON calificaciones.id_Materia = materia.id_Materia;

```

id_Alumno	Nombre_Alumno	Apellido_Alumno	id_Materia	Nombre_Materia	nota
1	Juan	López	1	Matemáticas	4.50
3	Pedro	García	1	Matemáticas	4.00
1	Juan	López	2	Programación	4.20
3	Pedro	García	2	Programación	3.90
2	María	Rodríguez	3	Finanzas	3.80
4	Laura	Ramírez	3	Finanzas	4.10
2	María	Rodríguez	4	Contabilidad I	4.00
4	Laura	Ramírez	4	Contabilidad I	4.30

```

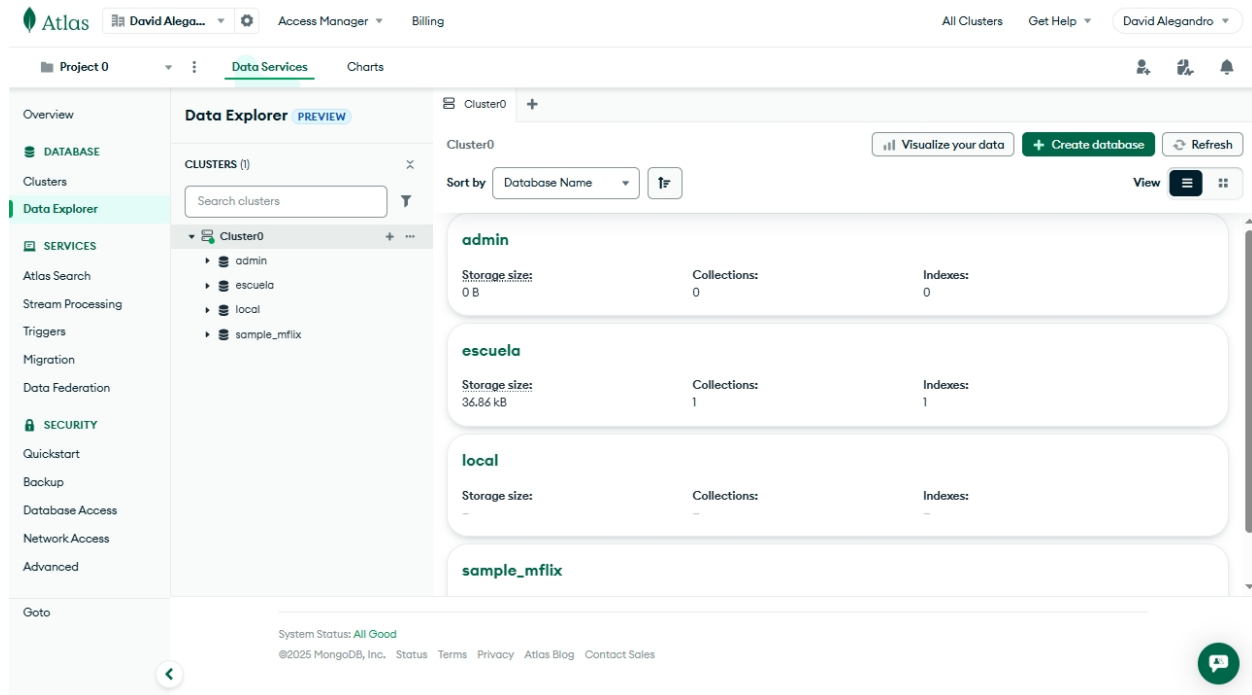
1 SELECT
2   profesor.id_Profesor,
3   profesor.Nombre,
4   profesor.Apellido,
5   profesor.Especialidad,
6   profesor.Cargo,
7   programa.Nombre AS Nombre_Programa
8 FROM profesor
9 INNER JOIN programa ON profesor.id_Curso = programa.id_Curso;

```

id_Profesor	Nombre	Apellido	Especialidad	Cargo	Nombre_Programa
1	Carlos	Pérez	Matemáticas	Titular	Sistemas
2	Luisa	Gómez	Informática	Titular	Sistemas
3	Ana	Martínez	Finanzas	Catedrática	Contabilidad

Base de datos NoSQL

Demostración del cluster con la base de datos creada:

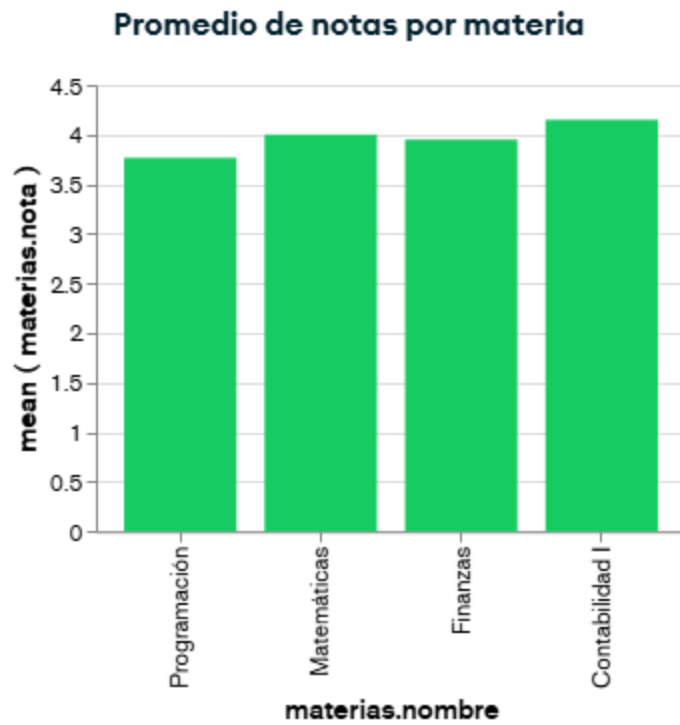


Ejemplo de datos en la base de datos:

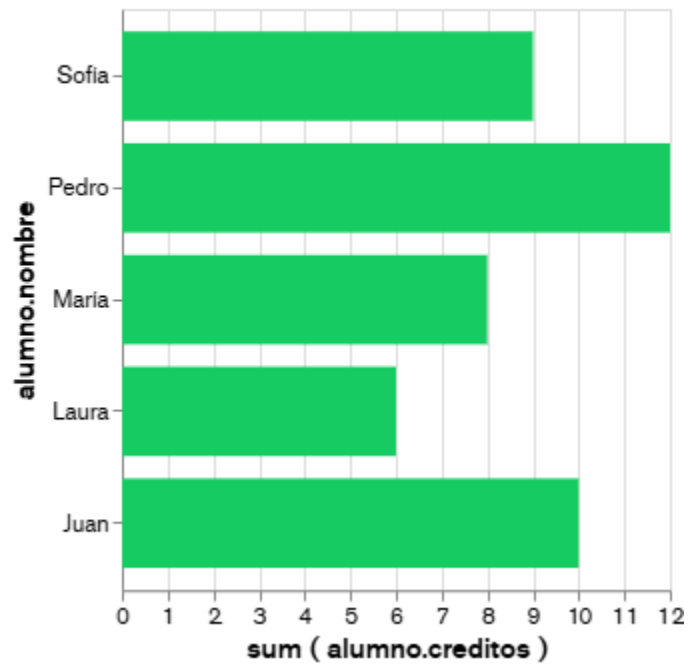
```
_id: ObjectId('6871b2034e760413d464bbe6')
  alumno: Object
    id: 1
    nombre: "Juan"
    apellido: "López"
    credits: 10
  programa: Object
  materias: Array (2)
```

```
_id: ObjectId('6871b25e4e760413d464bbf7')
  alumno: Object
    id: 2
    nombre: "María"
    apellido: "Rodríguez"
    credits: 8
  programa: Object
  materias: Array (2)
```

Ejemplos de una dashboard:



Cantidad de creditos por estudiante



Glosario:

- **UML (Unified Modeling Language - Lenguaje Unificado de Modelado):** Es un lenguaje gráfico y estándar para visualizar, especificar, construir y documentar los artefactos de un sistema de software. No es un lenguaje de programación, sino una forma de modelar y diseñar sistemas.
- **POO (Programación Orientada a Objetos):** Es un paradigma de programación que organiza el diseño del software en torno a "objetos", que son entidades que combinan datos (atributos) y comportamiento (métodos). Su objetivo es simular el mundo real, facilitando el desarrollo y mantenimiento de sistemas.
- **Objeto:** Es una instancia concreta de una **clase**. Representa una entidad del mundo real o conceptual con un estado (valores de sus atributos) y un comportamiento (acciones que puede realizar). Por ejemplo, si "Coche" es una clase, un "Ford Fiesta azul" es un objeto.
- **Clase:** Es una plantilla, un plano o una descripción abstracta que define las características (atributos) y los comportamientos (métodos) comunes a un conjunto de objetos. Es el tipo de un objeto.
- **Modelo:** En el contexto de software, un modelo es una representación simplificada de un sistema o un aspecto de este. Ayuda a comprender, analizar, diseñar y comunicar ideas sobre el sistema antes de su implementación. Un diagrama de clases UML es un tipo de modelo.
- **Atributo:** Una característica o propiedad de una **clase** que describe una cualidad del objeto. Se representa como una variable dentro de la clase que almacena un valor. Por ejemplo, `color` o `velocidad` para una clase `Coche`. Los alcances comunes son:
 - **Público (+):** Accesible desde cualquier lugar.
 - **Privado (-):** Accesible solo desde dentro de la propia clase.
 - **Protegido (#):** Accesible desde la propia clase y sus subclasses (clases que heredan de ella).
- **Parámetro:** Un valor que se le pasa a un método (o función) cuando se le llama. Actúa como un argumento para la operación, permitiendo que el método realice su acción de manera flexible. Por ejemplo, en un método `acelerar(velocidad)`, `velocidad` es un parámetro.
- **Agregación:** Es un tipo de relación de **asociación** que representa una relación de "parte-todo" o "tiene un", donde las "partes" pueden existir independientemente del "todo". Si el "todo" se destruye, las "partes" pueden permanecer. Se dibuja con un rombo vacío en el lado del "todo".
- **Herencia (Generalización/Especialización):** Es uno de los pilares de la POO. Permite que una **clase** (subclase o clase hija) adquiera los atributos y métodos de otra clase (superclase o clase padre). Establece una relación "es un tipo de". Por ejemplo, un "Perro" es un tipo de "Animal".
- **Polimorfismo:** Significa "muchas formas". En POO, permite que objetos de diferentes clases respondan al mismo mensaje (llamada de método) de maneras diferentes, siempre y cuando compartan una interfaz común. Esto se logra a menudo a través de la herencia y la sobreescritura de métodos.
- **Extensión:** En el contexto de los casos de uso (un concepto de UML relacionado con el comportamiento), una extensión describe cómo un caso de uso puede incluir opcionalmente el comportamiento de otro caso de uso bajo ciertas condiciones. En un sentido más general de software, puede referirse a añadir nueva funcionalidad.
- **Función (Método/Operación):** Una secuencia de instrucciones que realiza una tarea

específica. En POO, cuando una función pertenece a una clase, se le llama comúnmente **método** u **operación**. Define el comportamiento de los objetos de esa clase.

- **Relación:** En UML, una relación es una conexión o vínculo semántico entre dos o más elementos del modelo (como clases). Define cómo interactúan o se asocian entre sí. Ejemplos incluyen asociación, herencia, agregación, composición y dependencia.
- **Abstracción:** Es el proceso de ocultar los detalles de implementación complejos y mostrar solo la información esencial y relevante. En POO, se logra a través de clases abstractas, interfaces y encapsulamiento, permitiendo a los desarrolladores enfocarse en "qué" hace un objeto en lugar de "cómo" lo hace.
- **Sobrecarga (Overloading):** Permite definir múltiples métodos con el mismo nombre dentro de la misma clase, siempre y cuando tengan listas de parámetros diferentes (en número o tipo). Esto permite que una única operación se comporte de manera diferente según los argumentos que se le pasen.